



Distributed Systems

Fault Tolerance

credits for the slides to:

Timofei Istomin

Giuliano Mega

Luca Mottola

Davide Frey

Outline



- ◆ Introduction
- ◆ An example: client-server communication
- ◆ Reliable group communication
- ◆ Agreement in process groups
- ◆ Atomic commitment

Why be fault tolerant



- ◆ Tolerate faults — operate correctly (without **failing**) despite faults
- ◆ Why is this important?
- ◆ Your laptop:
 - ◆ $\text{Pr}[\text{CPU fault in (**full**) 30 days}] 0.5\%$ [Nightingale 2012]
 - ◆ that's a 1.64 years expectation for failure
- ◆ If you're Google, however...
 - ◆ ~ 1 million servers
 - ◆ ~ 5.000 faults/month
 - ◆ ~ 166 faults/day
 - ◆ ~ 7 /hour

Why be fault tolerant



- ◆ In August 2013, Google had 1~5 minutes of outage
 - ◆ Internet traffic dropped by 40%
 - ◆ estimated USD 500,000 revenue loss
- ◆ Google cannot afford downtime
- ◆ Distributed system mentality:
 - ◆ faults are a reality, plan for them

Metrics of Dependability



"Five nines" availability
99.999%; ~5min
downtime per year

1. Availability

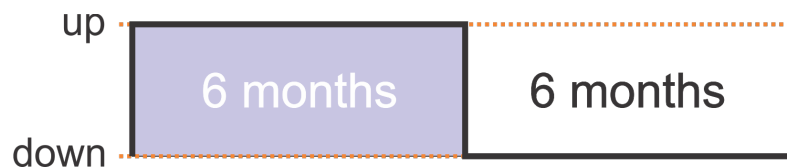
- ◆ pick a random instant t in time;
- ◆ what's the probability that the system is "up"?

2. Reliability

- ◆ system was brought up at time t_0 ;
- ◆ what's the probability it's still running by $t + t_0$?

Higher = more dependable

Can a system be reliable but not available?



Metrics of Dependability



3. Safety

- ◆ probability that the system will not suffer a catastrophic failure by time t
- ◆ e.g. a circuit burns in a life support system. What's the probability somebody ends up without oxygen?
- ◆ The lower, the safer

4. Maintainability:

- ◆ system goes down at time t_0
- ◆ what's the probability it's restored by $t + t_0$?
- ◆ Higher = more maintainable
- ◆ correlates to availability

5. Security*: ability to prevent unauthorized access/handling, not covered here.

Absolute dependability?



Do absolutely dependable systems exist?

Of course not!

Applying the theory



- ◆ The Fault Tolerance theory provides common terms to *reason* about faults and a set of *techniques* and *algorithms* to make a distributed system tolerate the faults
- ◆ The algorithms are **proven** to hold certain properties under certain conditions (e.g., types and number of simultaneous faults)
- ◆ Should unexpected faults occur – no guarantees are given, the system will probably fail

Applying the theory



- ◆ There are many types of systems with different requirements (reliability, cost, performance, etc.)
- ◆ It is up to the system designer to decide which techniques are appropriate for the system
 - ◆ Which faults are likely to occur
 - ◆ How critical are those faults
 - ◆ How much resources we can spend to tolerate them